機器學習實作系列｜選擇題題庫

共 92 題（第 1 週 12 題、第 2 週 16 題、第 3 週 16 題、第 5 週 16 題、第 6 週 16 題、第 7 週 16 題）。期中考範圍第 1–7 週。前半為題目，答案表與詳解在最後。

第 1 週 – 機器學習簡介

W1-1 代理人透過與環境互動、依獎勵訊號試錯學習，屬於哪一類學習？

An agent learns by trial and error from a reward signal while interacting with an environment. Which category of learning is this?

(A) 強化式學習
Reinforcement learning

(B) 監督式學習
Supervised learning

(C) 非監督式學習
Unsupervised learning

(D) 半監督學習
Semi-supervised learning

W1-2 分類與迴歸的差別是什麼？

What is the difference between classification and regression?

(A) 分類用監督式學習，迴歸用非監督式學習
Classification is supervised learning, regression is unsupervised learning

(B) 分類的特徵是離散的，迴歸的特徵是連續的
Classification has discrete features, regression has continuous features

(C) 分類的標籤是離散類別，迴歸的標籤是連續數值
Classification has discrete class labels, regression has continuous numeric labels

(D) 分類需要特徵縮放，迴歸不需要
Classification needs feature scaling, regression does not

W1-3 可訓練參數與超參數的差別是什麼？

What is the difference between trainable parameters and hyperparameters?

(A) 前者是整數，後者是浮點數
The former are integers, the latter floating-point numbers

(B) 前者訓練中自動學出，後者訓練前由人設定
The former are learned automatically during training, the latter set by a human beforehand

(C) 前者由人設定，後者訓練中自動學出
The former are set by a human, the latter learned automatically during training

(D) 兩者相同，只是不同框架的稱呼
They are the same thing, named differently by different frameworks

W1-4 特徵選取與特徵提取的差別是什麼？

What is the difference between feature selection and feature extraction?

(A) 前者在訓練前做，後者在訓練後做
The former is done before training, the latter after training

(B) 前者用於分類，後者用於迴歸
The former is for classification, the latter for regression

(C) 兩者都會產生全新的特徵欄位
Both produce entirely new feature columns

(D) 前者從原始特徵挑子集，後者組合轉換成新特徵
The former picks a subset of the original features, the latter combines and transforms them into new ones

W1-5 一個 epoch 指的是什麼？

What does one epoch mean?

(A) 訓練資料完整跑過一輪
One full pass over the training data

(B) 權重更新一次
One update of the weights

(C) 處理完一個批次的資料
Finishing one batch of data

(D) 完成一次完整的交叉驗證
Completing one full round of cross-validation

W1-6 過擬合與欠擬合分別對應什麼？

What do overfitting and underfitting each correspond to?

(A) 過擬合對應高偏差，欠擬合對應高變異
Overfitting means high bias, underfitting means high variance

(B) 兩者都對應高變異，差別在訓練集表現
Both mean high variance; they differ in training set performance

(C) 過擬合對應高變異，欠擬合對應高偏差
Overfitting means high variance, underfitting means high bias

(D) 兩者都對應高偏差，差別在測試集表現
Both mean high bias; they differ in test set performance

W1–7 No Free Lunch 定理告訴我們什麼？

What does the No Free Lunch theorem tell us?

(A) 越複雜的模型在所有任務上都越好
The more complex a model is, the better it does on every task

(B) 不存在一個分類器能在所有場景都表現最好
No single classifier performs best in every scenario

(C) 訓練資料越多，任何模型都會收斂到相同結果
With enough training data, every model converges to the same result

(D) 所有演算法的計算成本本質上相同
All algorithms cost essentially the same to compute

W1–8 梯度下降與隨機梯度下降的主要差別是什麼？

What is the main difference between gradient descent and stochastic gradient descent?

(A) 前者用於分類，後者用於迴歸
The former is for classification, the latter for regression

(B) 前者會收斂，後者不會收斂
The former converges, the latter does not

(C) 前者需要標籤，後者不需要
The former needs labels, the latter does not

(D) 前者每次用全部樣本算梯度，後者每次只用一筆估計
The former computes the gradient from all samples, the latter estimates it from one at a time

---

W1–9 反向傳播（backpropagation）做的是什麼事？

What does backpropagation do?

(A) 用連鎖律逐層計算損失對每個參數的梯度
It computes the gradient of the loss with respect to each parameter layer by layer, via the chain rule

(B) 直接決定每個參數的最佳值
It determines the optimal value of each parameter directly

(C) 把輸入資料逐層往前傳遞算出預測值
It passes the input data forward layer by layer to produce a prediction

(D) 調整學習率使訓練更快收斂
It tunes the learning rate so that training converges faster

W1–10 神經網路裡激活函數的作用是什麼？

What is the role of an activation function in a neural network?

(A) 把輸出正規化到 $[0, 1]$ 之間
It normalizes the output into the range $[0, 1]$

(B) 防止梯度在訓練中消失
It stops gradients from vanishing during training

(C) 引入非線性，讓網路能表達非線性關係
It introduces non-linearity, letting the network express non-linear relations

(D) 決定每一層要使用多少個神經元
It decides how many neurons each layer uses

W1–11 資料含有明顯離群值時，正規化與標準化該怎麼選？

When the data contains clear outliers, should normalization or standardization be chosen?

(A) 選正規化，它對離群值較穩健
Normalization, because it is more robust to outliers

(B) 選標準化，它對離群值較穩健
Standardization, because it is more robust to outliers

(C) 兩者受離群值影響相同
Outliers affect both of them equally

(D) 兩者都不適用，必須先刪除離群值
Neither applies; the outliers must be removed first

W1–12 從函式的觀點看，機器學習的「模型」必須滿足什麼條件？

Seen as a function, what condition must a machine learning “model” satisfy?

(A) 必須是連續且可微分的
It must be continuous and differentiable

(B) 必須能用一條數學公式明確寫出來
It must be expressible as an explicit mathematical formula

(C) 輸出必須落在有限的範圍內
Its output must fall within a bounded range

(D) 同一個輸入只能對應一個輸出
One input may map to one output only

第 2 週 – PLA 與 Python 基礎

W2-1 PLA 在某一筆樣本上分類正確時，會對權重做什麼？

When the PLA classifies a sample correctly, what does it do to the weights?

(A) 完全不更新
It does not update them at all

(B) 往正確方向小幅更新
It updates them slightly in the right direction

(C) 更新量減半
It halves the size of the update

(D) 把該筆樣本移出訓練集
It removes that sample from the training set

W2-2 Novikoff 收斂定理保證 PLA 在有限步內收斂，前提是什麼？

Novikoff's convergence theorem guarantees the PLA converges in a finite number of steps. On what premise?

(A) 學習率必須夠小
The learning rate must be small enough

(B) 特徵必須先標準化
The features must be standardized first

(C) 訓練資料必須線性可分
The training data must be linearly separable

(D) 樣本數必須大於特徵數
The number of samples must exceed the number of features

W2-3 Adaline 把感知器的階梯函數換成連續線性輸出，最關鍵的好處是什麼？

Adaline replaces the perceptron's step function with a continuous linear output. What is the key benefit?

(A) 輸出值被限制在 $[0, 1]$ 之間
The output value is confined to $[0, 1]$

(B) 損失函數變得可微分，能用梯度下降訓練
The loss becomes differentiable, so gradient descent can train it

(C) 可以直接處理多元分類
It can handle multi-class classification directly

(D) 不再需要設定學習率
A learning rate no longer has to be set

W2-4 在 $\boldsymbol{w}^T \boldsymbol{x} + b = 0$ 這個超平面中，$\boldsymbol{w}$ 與 b 各決定什麼？

In the hyperplane $\boldsymbol{w}^T \boldsymbol{x} + b = 0$, what does each of $\boldsymbol{w}$ and b determine?

(A) $\boldsymbol{w}$ 決定位置，b 決定方向
$\boldsymbol{w}$ sets the position, b sets the orientation

(B) 兩者共同決定方向，位置由資料決定
Together they set the orientation; the data sets the position

(C) $\boldsymbol{w}$ 決定維度，b 決定傾斜程度
$\boldsymbol{w}$ sets the dimension, b sets the degree of tilt

(D) $\boldsymbol{w}$ 決定法向量方向，b 決定離原點多遠
$\boldsymbol{w}$ sets the direction of the normal vector, b how far it sits from the origin

---

W2-5 One-versus-All（OvA）怎麼把二元分類器推廣到多元分類？

How does One-versus-All (OvA) extend a binary classifier to multi-class classification?

(A) 每個類別各訓練一個二元分類器，取信心最高者
Train one binary classifier per class and take the most confident one

(B) 每兩個類別訓練一個分類器，最後投票
Train one classifier for every pair of classes and vote at the end

(C) 訓練一個分類器，輸出層改成多個神經元
Train one classifier and give its output layer several neurons

(D) 把多個類別合併成兩個大類後再細分
Merge the classes into two large groups, then subdivide

W2-6 自適應學習率 $\eta t = \dfrac{\eta 0}{1 + \alpha t}$ 的用意是什麼？

What is the point of the adaptive learning rate $\eta t = \dfrac{\eta 0}{1 + \alpha t}$?

(A) 讓學習率隨損失下降而自動上升
It makes the learning rate rise automatically as the loss falls

(B) 避免模型過擬合訓練資料
It stops the model from overfitting the training data

(C) 前期步伐大快速接近，後期步伐小穩定收斂
Large steps approach quickly early on, small steps settle stably later

(D) 讓每個權重各自擁有不同的學習率
It gives every weight a learning rate of its own

W2-7 選用 Python 的 list、tuple、dict、set 時，主要看哪三個問題？

When choosing between Python's list, tuple, dict and set, which three questions matter most?

(A) 要不要排序、要不要去重、要不要轉型
Whether to sort, whether to deduplicate, whether to cast types

(B) 要不要改、要不要順序、要不要查得快
Whether it must change, whether order matters, whether lookup must be fast

(C) 資料量多大、型別是否一致、會不會跨執行緒
How large the data is, whether types are uniform, whether threads are involved

(D) 要不要巢狀、要不要序列化、要不要比較大小
Whether to nest, whether to serialize, whether to compare sizes

---

W2-8 Python 類別方法中的 `self` 代表什麼？

What does `self` stand for in a Python class method?

(A) 類別本身，而非實例
The class itself, rather than an instance

(B) 目前所在的模組
The module currently being executed

(C) 一個保留字，不能改名
A reserved word that cannot be renamed

(D) 呼叫這個方法的那個物件實例
The object instance on which this method was called

W2-9 PLA 遇到非線性可分的資料會發生什麼？

What happens when the PLA meets data that is not linearly separable?

(A) 錯誤次數永遠降不到零，權重持續震盪
The error count never reaches zero and the weights keep oscillating

(B) 自動停止並回報無解
It stops automatically and reports that no solution exists

(C) 收斂到誤差最小的那個超平面
It converges to the hyperplane with the smallest error

(D) 自動增加特徵維度直到可分
It adds feature dimensions automatically until the data is separable

---

W2-10 梯度下降的學習率設得太大，會發生什麼？

What happens when the learning rate of gradient descent is set too large?

(A) 收斂變慢，但最終仍會到達最低點
Convergence slows down, but the minimum is still reached in the end

(B) 模型過擬合訓練資料
The model overfits the training data

(C) 在谷壁兩側越彈越高，損失發散
It bounces ever higher off the walls of the valley and the loss diverges

(D) 梯度消失，權重不再更新
The gradient vanishes and the weights stop updating

W2-11 tuple 與 list 最主要的差別是什麼？

What is the main difference between a tuple and a list?

(A) tuple 只能存放同一種型別
A tuple may hold only one type of element

(B) tuple 建立後內容不可變更
A tuple cannot be changed once created

(C) tuple 沒有順序，無法用索引存取
A tuple has no order and cannot be indexed

(D) tuple 的查詢速度比 list 快得多
A tuple is looked up far faster than a list

W2-12 Adaline 訓練時用連續的線性輸出算損失，那實際要輸出類別時怎麼辦？

Adaline computes its loss from a continuous linear output during training. How does it emit an actual class?

(A) 直接四捨五入到最接近的類別編號
Round directly to the nearest class number

(B) 用階梯函數對淨輸入取符號
Take the sign of the net input with a step function

(C) 再訓練一個額外的分類器
Train one additional classifier on top

(D) 對淨輸入套用分類閾值，超過就判為正類
Apply a classification threshold to the net input; above it the class is positive

---

W2-13 下列程式會印出什麼？

What does the following code print?

```
a = [10, 20, 30, 40, 50]
print(a[1:3], a[-1], a[::-1][0])
```

(A) `[20, 30, 40] 50 10`
Prints `[20, 30, 40] 50 10`

(B) `[10, 20, 30] 50 50`
Prints `[10, 20, 30] 50 50`

(C) `[20, 30] 50 50`
Prints `[20, 30] 50 50`

(D) `[20, 30] 40 10`
Prints `[20, 30] 40 10`

W2-14 下列程式會印出什麼？

What does the following code print?

```
print(7 / 2, 7 // 2, 7 % 2, 2 ** 3)
```

(A) `3.5 3 1 8`
Prints `3.5 3 1 8`

(B) `3 3 1 6`
Prints `3 3 1 6`

(C) `3.5 3.5 1 8`
Prints `3.5 3.5 1 8`

(D) `3.5 3 1 6`
Prints `3.5 3 1 6`

W2-15 下列程式會印出什麼？

What does the following code print?

```
data = [3, 1, 3, 2, 1, 3]
counts = {}
for item in data:
    counts[item] = counts.get(item, 0) + 1
print(counts[3], len(set(data)))
```

(A) `2 6`
Prints `2 6`

(B) 第一次遇到 3 時丟出 `KeyError`
Raises `KeyError` the first time it meets 3

(C) `3 6`
Prints `3 6`

(D) `3 3`
Prints `3 3`

W2-16 下列程式會印出什麼？

What does the following code print?

```
nums = [1, 2, 3, 4, 5, 6]
print([n ** 2 for n in nums if n % 2 == 0])
```

(A) `[1, 9, 25]`
Prints `[1, 9, 25]`

(B) `[4, 16, 36]`
Prints `[4, 16, 36]`

(C) `[2, 4, 6]`
Prints `[2, 4, 6]`

(D) `[1, 4, 9, 16, 25, 36]`
Prints `[1, 4, 9, 16, 25, 36]`

第 3 週 – Adaline 與 NumPy、Pandas

W3-1 NumPy 判斷兩個陣列能否廣播，規則是什麼？

What is the rule by which NumPy decides whether two arrays can be broadcast together?

(A) 從最右邊維度起比較，每一對必須相等或其中一方為 1
Compare from the rightmost dimension; each pair must be equal or one of them must be 1

(B) 從最左邊維度起比較，每一對必須完全相等
Compare from the leftmost dimension; every pair must match exactly

(C) 兩個陣列的維度數必須相同才能廣播
The two arrays must have the same number of dimensions to broadcast

(D) 只要元素總數相同就能廣播
Any two arrays with the same total number of elements can broadcast

---

W3-2 聚合運算中的 `axis` 參數，指的是什麼？

What does the `axis` argument of an aggregation refer to?

(A) 結果要保留的維度
The dimension the result keeps

(B) 運算要沿著走訪的方向，與結果形狀無關
The direction the operation traverses, unrelated to the result shape

(C) 被壓縮掉的那個維度
The dimension that gets collapsed

(D) 陣列在記憶體中的排列順序
The order in which the array is laid out in memory

W3-3 `.loc` 與 `.iloc` 在切片行為上的差別是什麼？

How does slicing behave differently between `.loc` and `.iloc`?

(A) 兩者都包含尾端
Both include the endpoint

(B) `.loc` 包含尾端，`.iloc` 不包含
`.loc` includes the endpoint, `.iloc` does not

(C) `.loc` 不包含尾端，`.iloc` 包含
`.loc` excludes the endpoint, `.iloc` includes it

(D) 兩者都不包含尾端
Neither includes the endpoint

W3-4 對特徵做標準化之後，該特徵的分布會變成什麼？

After a feature is standardized, what does its distribution become?

(A) 常態分布
A normal distribution

(B) 均勻分布
A uniform distribution

(C) $[0, 1]$ 區間內的分布
A distribution within the interval $[0, 1]$

(D) 分布形狀不變，只是平均 0、標準差 1
The shape of the distribution is unchanged; only the mean becomes 0 and the standard deviation 1

W3-5 Mini-batch 梯度下降在全批次梯度下降與隨機梯度下降之間，扮演什麼角色？

What role does mini-batch gradient descent play between full-batch and stochastic gradient descent?

(A) 折衷：兼顧向量化效率與訓練穩定性
A compromise, keeping vectorized efficiency alongside stable training

(B) 完全取代兩者，沒有任何缺點
A full replacement for both, with no drawback at all

(C) 只適用於資料量極小的情況
Useful only when the dataset is extremely small

(D) 與全批次梯度下降等價，只是實作不同
Equivalent to full-batch gradient descent, differing only in implementation

---

W3-6 `AdalineSGD.partial_fit()` 這類線上學習的做法，關鍵是什麼？

What is the key to online learning of the kind `AdalineSGD.partial_fit()` performs?

(A) 每次呼叫都重新初始化權重再訓練
Reinitializing the weights and retraining on every call

(B) 把新資料與舊資料合併後完整重訓
Merging the new data with the old and retraining completely

(C) 不重新初始化權重，直接用新資料繼續更新
Not reinitializing the weights, but continuing to update from the new data

(D) 只更新偏差項，權重維持不變
Updating only the bias term, leaving the weights unchanged

W3-7 向量化為什麼比 Python 迴圈快？

Why is vectorization faster than a Python loop?

(A) 因為它自動使用多張 GPU
Because it uses several GPUs automatically

(B) 因為運算交給預先編譯的底層常式一次處理
Because the work is handed to precompiled low-level routines in one go

(C) 因為它會自動略過重複的計算
Because it automatically skips repeated computations

(D) 因為 NumPy 陣列佔用的記憶體比 list 小
Because a NumPy array occupies less memory than a list

W3-8 `concat` 與 `merge` 的差別是什麼？

What is the difference between `concat` and `merge`?

(A) `concat` 只能用於 `Series`，`merge` 只能用於 `DataFrame`
`concat` works only on a `Series`, `merge` only on a `DataFrame`

(B) `concat` 會去除重複列，`merge` 不會
`concat` drops duplicate rows, `merge` does not

(C) 兩者相同，只是參數寫法不同
They are the same; only the argument syntax differs

(D) `concat` 沿指定軸把資料疊起來，`merge` 依共同鍵值配對
`concat` stacks data along a chosen axis, `merge` matches rows by a shared key

---

W3-9 布林遮罩與花式索引的差別是什麼？

What is the difference between a boolean mask and fancy indexing?

(A) 前者用布林陣列篩選，後者用索引陣列指定位置
The former filters with a boolean array, the latter names positions with an index array

(B) 前者只能用於一維陣列，後者可用於多維
The former works on 1-D arrays only, the latter on multi-dimensional ones

(C) 前者回傳檢視，後者回傳複本
The former returns a view, the latter a copy

(D) 兩者相同，只是寫法不同
They are the same, written differently

W3-10 `np.newaxis` 的用途是什麼？

What is `np.newaxis` used for?

(A) 建立一個新的陣列
Creating a brand new array

(B) 把多維陣列攤平成一維
Flattening a multi-dimensional array into one dimension

(C) 插入一個長度為 1 的新軸，常用來觸發廣播
Inserting a new axis of length 1, typically to trigger broadcasting

(D) 交換兩個維度的順序
Swapping the order of two dimensions

W3-11 Adaline 訓練時用的線性激活 $\sigma(z) = z$，與最後分類用的閾值函數，關係是什麼？

How is the linear activation $\sigma(z) = z$ used while training Adaline related to the threshold function used for the final classification?

(A) 兩者是同一個函數，只是不同階段的稱呼
They are one function, named differently at different stages

(B) 前者保留連續誤差供訓練，後者只在預測時把輸出離散化
The former keeps a continuous error for training, the latter discretizes the output only at prediction time

(C) 前者用於預測，後者用於訓練
The former is for prediction, the latter for training

(D) 閾值函數會取代線性激活參與梯度計算
The threshold function replaces the linear activation in the gradient computation

---

W3-12 隨機梯度下降的損失曲線比全批次梯度下降毛躁，這件事的意義是什麼？

The loss curve of stochastic gradient descent is noisier than that of full-batch gradient descent. What does this signify?

(A) 代表實作有錯，正常應該平滑
It means the implementation is buggy; it should be smooth

(B) 代表學習率設得太大，必須調小
It means the learning rate is too large and must be reduced

(C) 代表模型正在過擬合
It means the model is overfitting

(D) 是單筆估計梯度的必然結果，抖動反而有機會跳出淺的局部低點
It follows inevitably from estimating the gradient one sample at a time, and the jitter can even escape shallow local minima

---

W3-13 下列程式印出的兩個形狀分別是什麼？

What are the two shapes printed by the following code?

```
import numpy as np
X = np.ones((3, 4))
print(X.sum(axis=0).shape, X.mean(axis=1).shape)
```

(A) `(3,) (4,)`
Prints `(3,) (4,)`

(B) `(4,) (3,)`
Prints `(4,) (3,)`

(C) `(1, 4) (3, 1)`
Prints `(1, 4) (3, 1)`

(D) `(4,) (4,)`
Prints `(4,) (4,)`

W3-14 下列程式的兩行輸出分別是什麼？

What are the two lines of output from the following code?

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[1, 0], [0, 1]])
print(A * B)
print(A @ B)
```

(A) 兩行相同，都是 `[[1 2] [3 4]]`
Both lines are the same: `[[1 2] [3 4]]`

(B) 兩行相同，都是 `[[1 0] [0 4]]`
Both lines are the same: `[[1 0] [0 4]]`

(C) 第一行 `[[1 2] [3 4]]`，第二行 `[[1 0] [0 4]]`
First line `[[1 2] [3 4]]`, second line `[[1 0] [0 4]]`

(D) 第一行 `[[1 0] [0 4]]`，第二行 `[[1 2] [3 4]]`
First line `[[1 0] [0 4]]`, second line `[[1 2] [3 4]]`

W3-15 依照廣播規則，下列兩個運算的結果分別是什麼？

According to the broadcasting rules, what are the results of the two operations below?

```
X = np.zeros((3, 4))
v = np.array([1, 2, 3, 4])
w = np.array([1, 2, 3])
X + v
X + w
```

(A) `X + v` 得到形狀 `(3, 4)`，`X + w` 報錯
`X + v` gives shape `(3, 4)`; `X + w` raises an error

(B) 兩者都得到形狀 `(3, 4)`
Both give shape `(3, 4)`

(C) `X + v` 報錯，`X + w` 得到形狀 `(3, 4)`
`X + v` raises an error; `X + w` gives shape `(3, 4)`

(D) 兩者都報錯，形狀必須完全相同
Both raise errors; the shapes must match exactly

W3-16 下列程式會印出什麼？

What does the following code print?

```
import pandas as pd
df = pd.DataFrame({"x": [10, 20, 30, 40]})
print(len(df.loc[0:2]), len(df.iloc[0:2]))
```

(A) `2 2`
Prints `2 2`

(B) `2 3`
Prints `2 3`

(C) `3 2`
Prints `3 2`

(D) `3 3`
Prints `3 3`

第 5 週 – 邏輯迴歸、Matplotlib 與 Seaborn

W5–1 某事件發生的機率是 0.8，它的勝算（odds）是多少？

An event has probability 0.8. What are its odds?

(A) 4
4

(B) 0.8
0.8

(C) 0.2
0.2

(D) 1.25
1.25

W5–2 邏輯迴歸用線性模型建模的對象是什麼？

What quantity does logistic regression model with a linear model?

(A) 機率 p 本身
The probability p itself

(B) 勝算 $\dfrac{p}{1-p}$
The odds $\dfrac{p}{1-p}$

(C) 對數勝算 $\ln\left(\dfrac{p}{1-p}\right)$
The log-odds $\ln\left(\dfrac{p}{1-p}\right)$

(D) 類別標籤 y
The class label y

W5–3 加上 sigmoid 之後，邏輯迴歸的決策邊界會變成什麼？

Once a sigmoid is applied, what does the decision boundary of logistic regression become?

(A) 變成非線性的曲面
It becomes a non-linear surface

(B) 仍然是線性的超平面
It is still a linear hyperplane

(C) 邊界形狀由資料分布決定
The shape of the boundary is set by the data distribution

(D) 沒有明確的決策邊界
There is no well-defined decision boundary

W5–4 「概似（likelihood）」是什麼的函數？

The likelihood is a function of what?

(A) 資料的函數，參數固定
A function of the data, with the parameters held fixed

(B) 預測正確率的函數
A function of the prediction accuracy

(C) 機率的函數，值域為 $[0, 1]$
A function of probability, with range $[0, 1]$

(D) 參數的函數，資料固定
A function of the parameters, with the data held fixed

W5–5 二元交叉熵損失對哪一種預測懲罰最重？

Which kind of prediction does binary cross-entropy loss punish most heavily?

(A) 很有自信卻答錯（例如預測 0.99 但真實標籤是 0）
Confident and wrong (predicting 0.99 when the true label is 0)

(B) 很有自信且答對
Confident and right

(C) 沒有自信但答對（預測 0.51）
Unconfident but right (predicting 0.51)

(D) 沒有自信且答錯（預測 0.49）
Unconfident and wrong (predicting 0.49)

W5–6 把特徵 $x1$ 改成 $|x1|$ 送進線性模型，是為了表達什麼？

Feeding $|x1|$ instead of $x1$ into a linear model expresses what?

(A) $x1$ 的正負號
The sign of $x1$

(B) $x1$ 的平方關係
A quadratic relation in $x1$

(C) 「離 0 太遠」這個概念
The notion of “too far from 0”

(D) $x1$ 與其他特徵的交互作用
The interaction between $x1$ and the other features

W5–7 Matplotlib 裡的 Figure 與 Axes 分別是什麼？
What are Figure and Axes in Matplotlib?

(A) Figure 是一條座標軸，Axes 是整張畫布
Figure is one coordinate axis, Axes is the whole canvas

(B) Figure 是整張畫布，Axes 是其中一個有座標軸的作圖區
Figure is the whole canvas, Axes is one plotting area on it with coordinate axes

(C) Figure 與 Axes 是同一個物件的別名
Figure and Axes are two names for the same object

(D) Figure 是資料容器，Axes 是樣式設定
Figure is the data container, Axes the style settings

W5–8 `plt.plot` 與 `plt.scatter` 的關鍵差別是什麼？
What is the key difference between `plt.plot` and `plt.scatter`?

(A) 前者只能畫折線，後者只能畫散點
The former draws only lines, the latter only points

(B) 前者用於連續資料，後者用於離散資料
The former is for continuous data, the latter for discrete data

(C) 前者較慢，後者較快
The former is slower, the latter faster

(D) 後者能逐點指定大小、顏色與透明度
The latter can set size, colour and transparency point by point

---

W5–9 直方圖的箱寬（bin width）設得太寬，會發生什麼？
What happens when the bin width of a histogram is set too wide?

(A) 可能把雙峰分布壓成單峰，掩蓋結構
A bimodal distribution may be flattened into one peak, hiding the structure

(B) 整張圖變成雜訊，看不出趨勢
The whole plot turns to noise and no trend is visible

(C) 樣本總數被改變
The total number of samples is changed

(D) 不影響圖形，只影響顯示速度
The plot is unaffected; only the rendering speed changes

W5–10 要用顏色編碼「溫度距平」這種有意義中點的量，該選哪一類色盤？
To colour-code a quantity with a meaningful midpoint, such as a temperature anomaly, which kind of palette should be chosen?

(A) 質性（qualitative）色盤
A qualitative palette

(B) 連續（sequential）色盤
A sequential palette

(C) 發散（divergent）色盤
A divergent palette

(D) 彩虹型色盤
A rainbow palette

W5–11 用 `plt.text` 標註時，把座標系指定為 `transAxes` 代表什麼？
When annotating with `plt.text`, what does setting the coordinate system to `transAxes` mean?

(A) 文字位置跟著資料座標跑
The text position follows the data coordinates

(B) 用子圖內 0 到 1 的相對位置定位
The text is placed by relative position from 0 to 1 within the subplot

(C) 相對整張畫布定位
The text is placed relative to the whole canvas

(D) 文字永遠置中於圖形正中央
The text is always centred in the middle of the plot

W5–12 什麼時候該用 Seaborn、什麼時候該回到 Matplotlib？
When should Seaborn be used, and when is it time to go back to Matplotlib?

(A) Seaborn 功能較強，應該全程使用
Seaborn is more capable and should be used throughout

(B) Matplotlib 較精確，應該全程使用
Matplotlib is more precise and should be used throughout

(C) 依資料量決定，大資料用 Seaborn
It depends on data size; large data calls for Seaborn

(D) 探索階段用 Seaborn，溝通階段用 Matplotlib
Seaborn for exploration, Matplotlib for communication

W5–13 邏輯迴歸訓練時最小化的損失函數是哪一個？
Which loss function does logistic regression minimize during training?

(A) 均方誤差（MSE），與 Adaline 相同
Mean squared error (MSE), the same as Adaline

(B) 分錯的樣本數，與感知器相同
The number of misclassified samples, the same as the perceptron

(C) 線性輸出 z 與標籤差值的絕對值
The absolute difference between the linear output z and the label

(D) 二元交叉熵，也就是負對數概似
Binary cross-entropy, i.e. the negative log-likelihood

W5–14 下列「模型—激活函數—輸出」的配對，哪一項正確？
Which of the following "model — activation function — output" pairings is correct?

(A) 感知器—sigmoid—正類的機率
Perceptron — sigmoid — probability of the positive class

(B) Adaline—恆等函數 $\sigma(z) = z$—任意實數，分類時再切門檻
Adaline — identity $\sigma(z) = z$ — any real number, thresholded for classification

(C) 邏輯迴歸—階梯函數—直接輸出 0 或 1
Logistic regression — step function — outputs 0 or 1 directly

(D) 邏輯迴歸—恆等函數 $\sigma(z) = z$—任意實數
Logistic regression — identity $\sigma(z) = z$ — any real number

---

W5–15 用 Adaline 的 MSE 做分類時，某個正類樣本（$y = 1$）的線性輸出是 $z = 5$，已經分得非常正確。接下來會發生什麼？
When Adaline's MSE is used for classification, a positive sample ($y = 1$) has linear output $z = 5$ and is already classified very confidently. What happens next?

(A) 損失為 0，參數不再因它而更新
The loss is 0, so it no longer updates the parameters

(B) 損失很小，只會讓參數微調
The loss is tiny and only nudges the parameters

(C) 損失仍有 $(1-5)^2 = 16$，決策邊界被拉向這個已分對的樣本
The loss is still $(1-5)^2 = 16$, pulling the boundary toward this correctly classified sample

(D) 它會被當成分錯，觸發感知器式的更新
It is treated as misclassified and triggers a perceptron-style update

---

W5–16 一張眼底攝影可能同時有白內障與高血壓病變。這種問題該用哪一種多類別做法？
A single fundus image may show both cataract and hypertensive changes at once. Which multi-class approach fits this problem?

(A) 一對其餘（OvR），每個類別各自輸出機率
One-vs-Rest (OvR), each class outputs its own probability

(B) Softmax，所有類別的機率總和固定為 1
Softmax, where all class probabilities sum to 1

(C) 只訓練一個二元邏輯迴歸，挑機率較高的一類
Train a single binary logistic regression and pick the more likely class

(D) 先用 softmax，再只輸出機率最大的那一類
Use softmax, then output only the most probable class

第 6 週 – Scikit-learn 機器學習

W6-1 Adaline 與感知器的關鍵差別在哪裡？

Where does Adaline differ crucially from the perceptron?

(A) 改用 sigmoid 函數把輸出壓到 0 與 1 之間
It uses a sigmoid to squash the output between 0 and 1

(B) 改成每看一筆資料就更新一次權重
It updates the weights once per sample seen

(C) 更新權重時看連續的線性輸出，並以平方誤差當損失
It updates the weights from a continuous linear output, with squared error as the loss

(D) 在損失函數外多加一項懲罰過大的權重
It adds a term outside the loss to penalize over-large weights

---

W6-2 關於 k 近鄰與 RBF 核支援向量機的計算成本，下列哪一項正確？

Which of the following is correct about the computational cost of k-nearest neighbours and of an RBF-kernel support vector machine?

(A) 兩者都是訓練昂貴、預測便宜，差別只在準確率
Both are expensive to train and cheap to predict; only accuracy differs

(B) k 近鄰訓練幾乎免費但預測昂貴，核支援向量機相反
k-NN is almost free to train but expensive to predict; the kernel SVM is the reverse

(C) k 近鄰訓練昂貴但預測便宜，核支援向量機相反
k-NN is expensive to train but cheap to predict; the kernel SVM is the reverse

(D) 兩者的訓練與預測都便宜，成本差異可以忽略
Both are cheap to train and to predict; the cost difference is negligible

---

W6-3 下列哪一個流程符合講義的黃金法則「先切分，再處理」？

Which of the following procedures follows the handout's golden rule, "split first, then process"?

(A) 先對整份資料標準化，再切分訓練集與測試集
Standardize the whole dataset, then split into training and test sets

(B) 先切分，再用整份資料的平均與標準差轉換兩邊
Split first, then transform both sides with the mean and standard deviation of the whole dataset

(C) 先切分，對訓練集與測試集分別呼叫 `fit_transform`
Split first, then call `fit_transform` separately on the training set and the test set

(D) 先切分，只用訓練集 `fit`，再用同一組轉換器 `transform` 測試集
Split first, `fit` on the training set only, then `transform` the test set with that same transformer

---

W6-4 關於訓練集與測試集的切分比例，下列哪一項正確？

Which of the following is correct about the split ratio between training set and test set?

(A) 測試集越小，對泛化誤差的估計越不準確
The smaller the test set, the less accurate the estimate of generalization error

(B) 測試集越小，對泛化誤差的估計越準確
The smaller the test set, the more accurate the estimate of generalization error

(C) 測試集越大，學習演算法能利用的資訊越多
The larger the test set, the more information the learning algorithm can use

(D) 固定用 80:20 最佳，與原始資料量沒有關係
A fixed 80:20 is always best, regardless of how much data there is

---

W6-5 用 `isnull()` 偵測缺值時，下列哪一項正確？

When detecting missing values with `isnull()`, which of the following is correct?

(A) 空白字串會被視為缺值，回傳 `True`
An empty string counts as missing and returns `True`

(B) `NaN` 與空白字串都不會被視為缺值
Neither `NaN` nor an empty string counts as missing

(C) 空白字串是有效資料，只有 `NaN`、`None`、`NaT` 回傳 `True`
An empty string is valid data; only `NaN`, `None` and `NaT` return `True`

(D) `isnull()` 無法偵測 `None`，需要另外處理
`isnull()` cannot detect `None`, which has to be handled separately

W6-6 關於類別標籤（class label）的編碼，下列哪一項正確？

Which of the following is correct about encoding class labels?

(A) 必須依類別出現的頻率由高到低指派整數
Integers must be assigned by class frequency, from highest to lowest

(B) 指派哪個整數給哪個字串標籤都無所謂
It makes no difference which integer is assigned to which string label

(C) 類別標籤必須做獨熱編碼才能餵給分類器
Class labels must be one-hot encoded before they can be fed to a classifier

(D) 類別標籤屬於順序型，要依領域知識映射
Class labels are ordinal and must be mapped using domain knowledge

---

W6-7 獨熱編碼產生的 0 / 1 虛擬欄位，接下來要怎麼處理？

What should be done next with the 0/1 dummy columns produced by one-hot encoding?

(A) 不需要再做特徵縮放，它本身已經是 0 與 1
No further feature scaling is needed; they are already 0 and 1

(B) 必須再做標準化，否則與其他欄位尺度不同
They must be standardized, or their scale will differ from the other columns

(C) 必須再做正規化，把它壓到 0 與 1 之間
They must be normalized, to squash them between 0 and 1

(D) 需不需要縮放取決於該欄相異值的個數
Whether scaling is needed depends on how many distinct values the column had

---

W6-8 資料含有極端離群值時，標準化與正規化的差別是什麼？

When the data contains extreme outliers, what is the difference between standardization and normalization?

(A) 正規化與標準化都會把離群值直接剔除
Normalization and standardization both discard outliers outright

(B) 標準化會把其餘資料擠進極小的子區間
Standardization squeezes the remaining data into a tiny sub-interval

(C) 兩者受離群值影響的程度相同，選哪個都一樣
Outliers affect both equally, so either choice is the same

(D) 正規化會把其餘資料擠進極小的子區間
Normalization squeezes the remaining data into a tiny sub-interval

---

W6-9 關於序列特徵選取（SFS 與 SBS）這類貪婪搜尋，下列哪一項正確？

Which of the following is correct about greedy searches such as sequential feature selection (SFS and SBS)?

(A) 它們會評估所有組合，因此保證找到最佳解
They evaluate every combination, so the optimum is guaranteed

(B) 它們的成本隨特徵數呈組合爆炸而不可行
Their cost explodes combinatorially with the feature count, making them infeasible

(C) 它們每階段做局部最優選擇，通常得到次佳解
They choose the local optimum at each stage, so the result is usually suboptimal

(D) 它們與窮舉搜尋的結果一致，只是速度較快
Their results match exhaustive search; they are merely faster

---

W6-10 一個參數較多的模型，訓練損失比參數較少的模型還高。這最可能是什麼問題？

A model with more parameters has a higher training loss than one with fewer. What is this most likely to be?

(A) 過擬合，應該加強正則化或者降維
Overfitting — regularize more strongly or reduce dimensionality

(B) 最佳化出了問題，應該換初始化或調學習率
An optimization problem — change the initialization or tune the learning rate

(C) 正常現象，參數越多訓練損失本來就越高
Normal — training loss naturally rises with more parameters

(D) 欠擬合，應該再繼續增加模型的參數量
Underfitting — keep adding parameters to the model

W6-11 為什麼 L_1 正則化常產生稀疏解，而 L_2 不會？

Why does L_1 regularization often produce sparse solutions while L_2 does not?

(A) 因為 L_1 的懲罰強度天生比 L_2 更重
Because the L_1 penalty is inherently heavier than the L_2 one

(B) 因為 L_1 的正則化項在原點不可微分
Because the L_1 regularization term is not differentiable at the origin

(C) 因為 L_1 會把小於門檻的權重直接設為零
Because L_1 sets any weight below a threshold straight to zero

(D) 因為 L_1 的約束區域是菱形，角落在座標軸上
Because the L_1 constraint region is a diamond, whose corners sit on the axes

---

W6-12 一個已經 fit 好的 Pipeline，呼叫 `predict()` 時內部會做什麼？

When `predict()` is called on an already-fitted Pipeline, what happens inside?

(A) 資料依序通過各轉換器的 `transform`，再交給估計器 `predict`

The data passes through each transformer's `transform` in turn, then goes to the estimator's `predict`

(B) 各轉換器先重新 `fit` 一次，再交給估計器 `predict`

Each transformer is re-`fit` once, then the data goes to the estimator's `predict`

(C) 各轉換器呼叫 `fit_transform`，再交給估計器 `predict`

Each transformer calls `fit_transform`, then the data goes to the estimator's `predict`

(D) 直接交給最後的估計器，略過前面所有的轉換器

The data goes straight to the final estimator, skipping every transformer before it

---

W6-13 下列程式哪裡有問題？

What is wrong with the following code?

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.fit_transform(X_test)
```

(A) 第 4 行對測試集重新 fit，用了測試集自己的統計量，造成資料洩漏

Line 4 re-fits on the test set, using its own statistics and causing data leakage

(B) 應該先對整份資料標準化，再切成訓練集與測試集

The whole dataset should be standardized first, then split into training and test sets

(C) `fit_transform` 只能用在估計器上，轉換器要分開呼叫

`fit_transform` only works on estimators; transformers must call the steps separately

(D) 沒有問題，兩份資料各自標準化才公平

Nothing is wrong; standardizing each set separately is fairer

---

W6-14 下列程式最後印出的 `coef_` 與 `intercept_` 分別是什麼？

What are the `coef_` and `intercept_` printed at the end of the following code?

```
from sklearn.linear_model import Perceptron
ppn = Perceptron(eta0=0.1, random_state=1)
ppn.fit(X_train_std, y_train)
print(ppn.coef_, ppn.intercept_)
```

(A) 兩者都是建構子傳進去的超參數

Both are hyperparameters passed to the constructor

(B) `coef_` 是學習率，`intercept_` 是迭代次數

`coef_` is the learning rate; `intercept_` is the number of iterations

(C) `coef_` 是學到的權重，`intercept_` 是偏置項

`coef_` is the learned weights; `intercept_` is the bias

(D) `coef_` 是各特徵的平均，`intercept_` 是標準差

`coef_` is each feature's mean; `intercept_` is the standard deviation

W6-15 下列關於「哪些模型需要特徵縮放」的敘述，哪一項正確？

Which statement about which models need feature scaling is correct?

(A) 決策樹與隨機森林都必須先標準化，否則切分點會錯

Decision trees and random forests must be standardized first or their split points go wrong

(B) KNN 與 SVM 依賴距離或間隔，應該先縮放

KNN and SVM rely on distances or margins, so features should be scaled first

(C) 邏輯迴歸不受特徵尺度影響，不必縮放

Logistic regression is unaffected by feature scale, so no scaling is needed

(D) 所有模型都一樣，縮放只是為了讓訓練跑快一點

All models are the same; scaling only makes training faster

W6-16 下列程式中 `stratify=y` 的作用是什麼？

What does `stratify=y` do in the following code?

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y)
```

(A) 固定亂數種子，讓每次切分的結果都相同
Fixes the random seed so every split comes out the same

(B) 讓測試集佔全部資料的 30%
Makes the test set 30% of all the data

(C) 先依標籤 y 排序，再切出前後兩段
Sorts by the label y first, then cuts off the front and back parts

(D) 讓訓練集與測試集的類別比例與原始資料相同
Keeps the class proportions in both sets the same as in the original data

第 7 週 – 資料前處理、特徵工程與模型選擇評估

W7-1 「模型選擇」在講義裡有寬窄兩義，下列哪一項正確？

"Model selection" carries a narrow and a broad sense in the handout. Which of the following is correct?

(A) 狹義指比較不同演算法，廣義指調同一個演算法的參數
The narrow sense is comparing algorithms, the broad sense is tuning the parameters of one algorithm

(B) 狹義指對指定演算法調參，廣義指比較不同演算法
The narrow sense is tuning the parameters of a given algorithm, the broad sense is comparing algorithms

(C) 兩者其實是同一件事，只是不同教材的說法差異
They are really the same thing, worded differently by different textbooks

(D) 狹義用巢狀交叉驗證，廣義用單層交叉驗證即可
The narrow sense needs nested cross-validation, the broad sense needs only a single loop

---

W7-2 保留法交叉驗證（holdout）最主要的弱點是什麼？

What is the main weakness of holdout cross-validation?

(A) 它沒有把測試集獨立出來，會高估泛化效能
It sets no test set aside, so it overestimates generalization performance

(B) 它需要訓練 k 個模型，計算成本相對太高
It has to train k models, which costs relatively too much

(C) 結果對「訓練集與驗證集怎麼分」很敏感
The result is very sensitive to how the training and validation sets happen to be split

(D) 它無法用在分類問題上，只適合迴歸問題
It cannot be used on classification problems, only on regression

---

W7-3 資料量與 k 折交叉驗證的 k 值，兩者的關係是什麼？

How is the amount of data related to the value of k in k-fold cross-validation?

(A) 資料量小用大的 k，資料量大用小的 k
Little data calls for a large k, plentiful data for a small k

(B) 資料量小用小的 k，資料量大用大的 k
Little data calls for a small k, plentiful data for a large k

(C) k 一律固定為 10，與資料量沒有關係
k is always fixed at 10, regardless of the amount of data

(D) k 要等於類別數，才能維持每折的比例
k must equal the number of classes to keep the proportions in each fold

---

W7-4 用 k 折交叉驗證找到滿意的超參數之後，下一步該做什麼？

After k-fold cross-validation has found satisfactory hyperparameters, what is the next step?

(A) 直接從那 k 個模型中挑分數最高的一個上線
Deploy whichever of those k models scored highest

(B) 把 k 個模型的預測結果平均，當作最終模型
Average the predictions of the k models and use that as the final model

(C) 用測試集再調一次超參數，確保分數是最佳的
Tune the hyperparameters once more on the test set, to make sure the score is the best

(D) 用完整的訓練集以該組超參數重新訓練一次
Retrain once on the full training set with that set of hyperparameters

---

W7-5 學習曲線上訓練與驗證兩條線都低、而且已經貼合，下一步該做什麼？

On a learning curve, both the training and validation lines are low and have already converged. What should be done next?

(A) 蒐集更多訓練資料，把兩條線一起往上拉
Collect more training data, to pull both lines up together

(B) 增加模型的複雜度，或者調弱正則化強度
Increase the model complexity, or weaken the regularization strength

(C) 提高正則化強度，並且減少使用的特徵數
Raise the regularization strength and use fewer features

(D) 什麼都不用做，這已經是理想的擬合狀態
Nothing at all; this is already an ideal fit

W7-6 看驗證曲線挑超參數時，應該選哪一個值？

When picking a hyperparameter from a validation curve, which value should be chosen?

(A) 訓練分數最高的那個值
The value with the highest training score

(B) 訓練與驗證分數差距最大的那個值
The value where training and validation scores differ the most

(C) 驗證分數見頂的那個值
The value where the validation score peaks

(D) 最右端、兩條線都還在上升的那個值
The rightmost value, where both lines are still rising

W7-7 同樣的評估預算下，隨機搜尋通常比格狀搜尋划算，原因是什麼？

For the same evaluation budget, why is random search usually a better deal than grid search?

(A) 隨機搜尋保證能找到全域最佳的超參數組合
Random search guarantees the globally optimal hyperparameter combination

(B) 隨機搜尋每一次評估的計算成本比較低廉
Each evaluation of random search costs less to compute

(C) 隨機搜尋會自動淘汰表現較差的候選組合
Random search automatically discards the weaker candidate combinations

(D) 真正重要的超參數通常只有少數幾個
Only a few hyperparameters usually matter at all

W7-8 巢狀交叉驗證的兩層迴圈各自負責什麼？

What is each of the two loops of nested cross-validation responsible for?

(A) 外層負責評估效能，內層負責挑超參數
The outer loop evaluates performance, the inner loop picks hyperparameters

(B) 外層負責挑超參數，內層負責評估效能
The outer loop picks hyperparameters, the inner loop evaluates performance

(C) 兩層都挑超參數，最後取兩次結果的平均
Both loops pick hyperparameters, and the two results are averaged at the end

(D) 兩層都評估效能，藉此縮小估計值的變異
Both loops evaluate performance, which narrows the variance of the estimate

---

W7-9 F1 分數取精確率與召回率的調和平均而非算術平均，為什麼？

Why does the F1 score take the harmonic rather than the arithmetic mean of precision and recall?

(A) 因為調和平均的計算比較省時間
Because the harmonic mean is faster to compute

(B) 因為調和平均的值域剛好落在 0 與 1 之間
Because the harmonic mean happens to fall between 0 and 1

(C) 因為調和平均對兩者中較小的那個特別敏感
Because the harmonic mean is especially sensitive to whichever of the two is smaller

(D) 因為精確率與召回率的單位不同，不能相加
Because precision and recall have different units and cannot be added

---

W7-10 把決策門檻調鬆（更容易判為陽性），精確率與召回率會怎麼變化？

If the decision threshold is loosened (making a positive call easier), how do precision and recall change?

(A) 精確率上升、召回率下降
Precision rises, recall falls

(B) 召回率上升、精確率下降
Recall rises, precision falls

(C) 兩者同時上升，因為判為陽性的樣本變多了
Both rise, because more samples are called positive

(D) 兩者都不會變，門檻只影響整體的正確率
Neither changes; the threshold only affects overall accuracy

W7-11 類別不平衡時，為什麼 MCC 比 F1 更不容易被誤導？

Under class imbalance, why is MCC harder to mislead than F1?

(A) 因為 MCC 的值域是 -1 到 1，解析度比較高
Because MCC ranges from -1 to 1, giving it finer resolution

(B) 因為 MCC 不受決策門檻位置的影響
Because MCC is unaffected by where the decision threshold sits

(C) 因為 MCC 只看少數類別的表現，忽略多數類
Because MCC looks only at the minority class and ignores the majority

(D) 因為 MCC 同時用上混淆矩陣的四格數字
Because MCC uses all four cells of the confusion matrix at once

---

W7-12 在 Pipeline 裡指定某個估計器的超參數（例如邏輯斯迴歸的 `C`），語法是什麼？

What is the syntax for setting a hyperparameter of an estimator inside a Pipeline (the `C` of a logistic regression, say)?

(A) 用雙底線連接步驟名稱與參數名稱
Join the step name and the parameter name with a double underscore

(B) 用單底線連接步驟名稱與參數名稱
Join the step name and the parameter name with a single underscore

(C) 直接寫參數名稱，Pipeline 會自動對應
Write the parameter name alone; the Pipeline matches it automatically

(D) 先從 Pipeline 取出該步驟的物件再單獨設定
Pull the step's object out of the Pipeline and set it separately

---

W7-13 下列程式最後印出的數字是多少？

What number does the following code print at the end?

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(estimator=pipe_lr, X=X_train, y=y_train,
                         cv=10, n_jobs=1)
print(len(scores))
```

(A) 1，因為回傳的是平均分數
1, because it returns the mean score

(B) 9，因為每次用 9 折訓練
9, because each round trains on 9 folds

(C) 與訓練樣本數相同
The same as the number of training samples

(D) 10，每一折各一個驗證分數
10, one validation score per fold

W7-14 執行下列 `gs.fit(...)` 時，交叉驗證階段總共會訓練幾個模型（不含最後用最佳參數重新訓練的那一次）？

When `gs.fit(...)` below runs, how many models are trained during cross-validation (excluding the final refit with the best parameters)?

```
param_grid = {"svc__C": [0.1, 1, 10, 100],
              "svc__gamma": [0.01, 0.1, 1]}
gs = GridSearchCV(pipe_svc, param_grid, cv=5)
gs.fit(X_train, y_train)
```

(A) 60 個
60 models

(B) 12 個
12 models

(C) 35 個
35 models

(D) 20 個
20 models

W7-15 接續上一題，下列程式在做什麼？

Continuing from the previous question, what does the following code do?

```
clf = gs.best_estimator_
clf.fit(X_train, y_train)
print(clf.score(X_test, y_test))
```

(A) 在測試集上再做一次格狀搜尋，挑出更好的參數
Runs another grid search on the test set to find better parameters

(B) 印出最佳參數組合在交叉驗證中的平均分數
Prints the mean cross-validation score of the best parameter combination

(C) 取出最佳參數的模型，在測試集上做最後一次評估
Takes the model with the best parameters and evaluates it once on the test set

(D) 用測試集繼續訓練模型，讓分數再往上提升
Keeps training the model on the test set to push the score higher

---

W7-16 下列程式會印出什麼（四捨五入到小數第二位）？

What does the following code print (rounded to two decimal places)?

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 1, 1, 0, 0, 0]
y_pred = [1, 0, 0, 1, 0, 0]
print(precision_score(y_true, y_pred), recall_score(y_true, y_pred))
```

(A) `0.33 0.50`
Prints `0.33 0.50`

(B) `0.50 0.33`
Prints `0.50 0.33`

(C) `0.50 0.50`
Prints `0.50 0.50`

(D) `0.33 0.33`
Prints `0.33 0.33`

答案表

第 1 週	1 A	2 C	3 B	4 D	5 A	6 C	7 B	8 D	9 A	10 C	11 B	12 D				
第 2 週	1 A	2 C	3 B	4 D	5 A	6 C	7 B	8 D	9 A	10 C	11 B	12 D	13 C	14 A	15 D	16 B
第 3 週	1 A	2 C	3 B	4 D	5 A	6 C	7 B	8 D	9 A	10 C	11 B	12 D	13 B	14 D	15 A	16 C
第 5 週	1 A	2 C	3 B	4 D	5 A	6 C	7 B	8 D	9 A	10 C	11 B	12 D	13 D	14 B	15 C	16 A
第 6 週	1 C	2 B	3 D	4 A	5 C	6 B	7 A	8 D	9 C	10 B	11 D	12 A	13 A	14 C	15 B	16 D
第 7 週	1 B	2 C	3 A	4 D	5 B	6 C	7 D	8 A	9 C	10 B	11 D	12 A	13 D	14 A	15 C	16 B

詳解

第 1 週 – 機器學習簡介

W1–1　答 A

強化式學習的特徵是「互動、試錯、依獎勵調整策略」，目標是讓累積的總獎勵最大化，資料不是事先備好的成對輸入與標籤。

第二項是最容易誤選的：獎勵訊號看起來像標籤，但兩者性質不同——標籤直接告訴模型「正確答案是什麼」，獎勵只告訴代理人「這一步好不好」，而且往往要等好幾步之後才知道。

考的觀念：有沒有標籤、標籤從哪裡來，是四種學習方式的分界。

W1–2　答 C

兩者都是監督式學習，差別在**標籤**的型態：離散類別是分類，連續數值是迴歸。

第二項把標籤講成特徵，是最常見的混淆——特徵是離散或連續，與任務屬於分類還是迴歸無關。判斷任務類型永遠先看「要預測的那一欄長什麼樣」。

考的觀念：看標籤，不是看特徵。

W1–3　答 B

可訓練參數（例如權重）是訓練過程中從資料學出來的；超參數（例如學習率、樹深、k 值）必須在訓練開始前由人決定。

第三項把兩者對調，是最容易記反的一組。一個實用的判準：訓練跑完之後值會變的是可訓練參數，值不會變的是超參數。

考的觀念：誰在訓練過程中被更新，誰就是可訓練參數。

W1–4　答 D

特徵選取是從既有欄位中挑一個子集，欄位本身不變，因此保留了原來的物理意義；特徵提取則是把特徵組合轉換成新的表示（例如 PCA），欄位本身已經改變，新特徵通常不再有直觀意義。

第三項只對一半——會產生全新欄位的只有特徵提取。這個差別在需要對人解釋模型時特別重要：選取後還能說「是因為這一欄」，提取後就不行了。

考的觀念：欄位有沒有被重新組合，是兩者的分界。

W1–5　答 A

一個 epoch 是訓練資料被完整看過一遍。

第二項是最容易混淆的：一個 epoch 裡的權重更新次數取決於訓練方式——批次梯度下降一個 epoch 只更新一次，隨機梯度下降則每看一筆就更新一次，一個 epoch 會更新樣本數那麼多次。兩者的 epoch 定義相同，更新次數卻差很多。

考的觀念：epoch 數的是「資料被看過幾遍」，不是「權重被更新幾次」。

W1–6　答 C

過擬合是模型把訓練資料的細節（含雜訊）也記住了，訓練集表現好但驗證與測試集明顯變差，表現為高變異；欠擬合是模型太簡單，連訓練資料本身的規律都學不好，表現為高偏差。

第一項把兩者對調，是最容易記反的一組。可以從字面接：變異衡量的是「換一份訓練資料結果會差多少」，記太多細節的模型自然變異大。

考的觀念：訓練集表現好不好，是區分兩者的第一個問題。

W1–7　答 B

沒有哪一個演算法能在所有問題上勝出，因此實務上要針對手上的問題比較多種演算法，而不是預設某個模型最強。

第一項正是這條定理要否定的直覺。第三項也不對：不同演算法的歸納偏好不同，資料再多也不會讓它們收斂到同一個解。

考的觀念：模型選擇是任務相依的，沒有通用最佳解。

W1–8　答 D

梯度下降每更新一次得掃完整份資料，梯度準確但慢；隨機梯度下降每看一筆就更新一次，梯度只是估計值、損失曲線比較毛躁，但更新快得多，也天然支援線上學習。

第二項說得太絕對：隨機梯度下降照樣會收斂，只是會在最低點附近抖動而不是穩穩停住——而這個抖動反而有機會跳出淺的局部低點。

考的觀念：差別在「一次更新用了多少資料」。

W1–9　答 A

反向傳播是一套**計算梯度**的演算法：用連鎖律，從輸出層往輸入層逐層求出損失對每個參數的偏微分。拿到梯度之後怎麼更新參數，那是最佳化器（梯度下降等）的工作。

第二項是最常見的誤解，把「算梯度」直接當成「找到最佳解」。第三項描述的是前向傳播，方向剛好相反。

考的觀念：反向傳播只負責算梯度，不負責更新。

W1–10　答 C

沒有激活函數的話，不論疊幾層，線性運算的合成仍然是線性的，整個網路等價於一層——激活函數提供的非線性才是深度有意義的前提。

第一項只描述了某些激活函數（例如 sigmoid）的副作用，不是它存在的理由；ReLU 的輸出就沒有上界。第二項的因果甚至相反：sigmoid 這類激活函數本身就是梯度消失的來源之一。

考的觀念：沒有非線性，疊再多層也只是一層。

W1–11　答 B

正規化 $x' = \dfrac{x - \min}{\max - \min}$ 的分母是極差，單一個極端值就能把其餘資料擠進極小的區間；標準化 $x' = \dfrac{x - \text{mean}}{\text{std}}$ 用平均與標準差，受單點影響小得多，因此較穩健。

第四項把手段當成前提——離群值不一定是錯誤資料，醫療與金融場景中它往往正是最重要的樣本，不能為了縮放方便就刪掉。

考的觀念：看分母用了什麼統計量，就知道它對極值敏不敏感。

W1–12　答 D

模型就是待求的函式 $f(\cdot)$，函式的定義要求給定輸入 x 唯一對應一個輸出 $f(x)$——同一個 x 不能對應兩個不同的結果。

第一項與第二項描述的是某些模型恰好具備的性質，不是函式的必要條件：決策樹既不連續也不可微，隨機森林更沒有一條簡潔的公式，它們仍然是合格的模型。

考的觀念：模型的本質是「輸入到輸出的唯一對應」。

第 2 週 – PLA 與 Python 基礎

W2–1　答 A

感知器學習規則是「猜對不動，猜錯才更新」：$w \leftarrow w + \eta y x$、$b \leftarrow b + \eta y$ 只在分類錯誤時執行。

第二項是最容易誤選的，因為多數學習演算法（例如 Adaline、邏輯斯迴歸）確實每一筆都更新——它們看的是連續的誤差量而非對錯。感知器只看對錯，猜對時誤差為零，自然沒有更新量。

考的觀念：只看對錯的模型，答對就沒有訊號可學。

W2–2　答 C

只要存在一個超平面能把兩類完全分開，PLA 就保證在有限步內找到一組完全分類正確的 $\boldsymbol{w}$ 與 b。

第一項是常見的混淆：學習率影響的是收斂速度與路徑，不是收斂與否——感知器的更新方向只由分錯的樣本決定，η 只縮放步長。標準化（第二項）同樣有助於訓練，但不是定理的前提。

考的觀念：收斂保證來自資料的可分性，不是超參數的設定。

W2-3 答 B

階梯函數不可微分，損失對權重的梯度無從算起，只能用「猜錯才修正」這種規則式更新。換成連續線性輸出後，均方誤差成為可微的碗狀曲面，標準梯度下降就能派上用場，訓練也從「有沒有錯」升級成「錯了多少」。

第一項描述的是 sigmoid 的性質，Adaline 的線性輸出並沒有上下界。第四項剛好相反：改用梯度下降之後，學習率反而成為關鍵超參數。

考的觀念：可微分是梯度下降的入場券。

W2-4 答 D

$\boldsymbol{w}$ 是超平面的法向量，決定它的朝向；b 則把這個平面平移，決定它離原點的距離。二維特徵空間裡，$\boldsymbol{w}$ 決定直線的斜率，b 決定截距。

第一項把兩者對調。少了 b 的模型會被迫讓決策邊界通過原點，表達能力因此受限——這正是偏差項不能省的理由。

考的觀念：法向量定方向，偏差項定位置。

W2-5 答 A

OvA 為每一個類別訓練一個「這類 vs 其餘」的二元分類器，預測時把樣本丟給全部分類器，取信心最高的那個作為答案。

第二項描述的是 One-versus-One，是另一種常見策略，分類器數量是 $\binom{k}{2}$ 而非 k 個。scikit-learn 多數分類器預設就以 OvA 支援多類別。

考的觀念：OvA 的分類器數量等於類別數。

W2-6 答 C

t 是 epoch 數，分母隨訓練進行而變大，學習率因此單調遞減：前期用大步伐快速逼近最低點附近，後期縮小步伐避免在谷底來回震盪。

第四項描述的是 AdaGrad、Adam 這類「每個參數各自調整」的最佳化器，與這裡單純隨時間衰減的排程不同。第二項也不對：學習率排程處理的是最佳化，不是泛化。

考的觀念：這是隨時間衰減的排程，不是隨參數調整的最佳化器。

W2-7 答 B

可不可變（list 可變、tuple 不可變）、有沒有順序（list 與 tuple 有，dict 與 set 無序地存放）、查詢快不快（dict 與 set 用雜湊，成員查詢接近常數時間）——這三個問題就足以選定容器。

第一項的「要不要去重」只對應到 set 這一個特性，涵蓋不了另外三種。

考的觀念：用可變性、順序性、查詢成本三個軸來分類。

W2-8 答 D

`self` 是實例方法的第一個參數，指向呼叫該方法的物件實例；`self.w` 這類寫法存取的是該實例自己的屬性。

第一項是最常見的誤解——代表類別本身的是 `cls`，出現在 `@classmethod` 裡。第三項也不對：`self` 只是慣例名稱、不是保留字，改成別的名字程式照樣會動，但沒有人會這麼做。

考的觀念：`self` 是慣例，不是語法強制。

W2-9 答 A

感知器只在分錯時更新，只要還有分錯的樣本就會繼續改權重。資料非線性可分時永遠存在分錯的樣本，於是權重一直被推來推去、錯誤次數降不到零。

第三項是最容易誤選的：PLA 沒有任何「最小化誤差」的目標函數可言，它不會停在最好的那個超平面上，只會停在你設定的迭代上限。這正是 Adaline 改用可微損失函數的動機之一。

考的觀念：沒有損失函數，就沒有「最佳解」這個概念。

W2-10 答 C

損失曲面是碗狀的，步伐太大時一步就跨過谷底衝到對側更高的位置，下一步又跨回來且更高——損失因此不降反升而發散。

第一項描述的是學習率太**小**的情況。這也是為什麼訓練的第一個檢查項目永遠是「損失有沒有穩定下降」，而不是急著看準確率：損失都發散了，準確率沒有任何意義。

考的觀念：先確認損失穩定下降，再談準確率。

W2-11 答 B

tuple 是不可變容器，建立之後不能新增、刪除或改寫元素；list 則可以。

第三項把它跟 set 搞混了——tuple 有順序，照樣能用索引存取。第一項也不對，tuple 可以混放不同型別。不可變帶來的實際好處是它可雜湊，因此能當作 dict 的鍵，list 不行。

考的觀念：不可變性帶來的是「可當字典鍵」這個能力。

W2-12 答 D

訓練與預測用的是同一個淨輸入 z，但用途不同：訓練時 z 直接進損失函數算梯度，預測時才對 z 套一個閾值轉成類別（本週實作取 $z \geq 0.5$ 判為 1）。

第三項不必要——閾值只是後處理，不需要額外模型。這個「訓練用連續值、預測才離散化」的分工，後面的邏輯斯迴歸也是同一套做法，差別只在中間多套了一層 sigmoid。

考的觀念：連續輸出負責學習，閾值負責決策。

W2-13 答 C

索引從 0 起算，`a[1:3]` 取索引 1、2，`stop` 不包含，所以是 `[20, 30]`；`a[-1]` 是最後一個元素 50；`a[::-1]` 把整個串列反轉成 `[50, 40, 30, 20, 10]`，再取第 0 個仍是 50。

選 (A) 的人把 `stop` 當成包含在內；選 (B) 的人把索引當成從 1 起算；選 (D) 的人把 `-1` 當成倒數第二個，又把反轉後的第 0 個當成原本的第 0 個。

考的觀念：切片不含 `stop`、負索引從尾端算、`[::-1]` 反轉。

W2-14 答 A

`/` 一律回傳 float，`7 / 2` 是 `3.5`；`//` 才是整數除法，得 `3`；`%` 是取餘數，得 `1`；`**` 是次方，`2 ** 3` 是 8。

選 (B) 的人把 `/` 當成 C 語言的整數除法，又把 `**` 當成乘法；選 (C) 的人分不清 `/` 與 `//`；選 (D) 的人把 `2 ** 3` 算成 2×3。

考的觀念：`/`、`//`、`%`、`**` 四個算術運算子的差別。

W2-15 答 D

這段程式在做「計數」：`counts.get(item, 0)` 在鍵不存在時回傳預設值 0，所以第一次遇到某個數字也能直接加 1，不會出錯。3 在串列裡出現 3 次，`counts[3]` 是 3。`set(data)` 去掉重複後剩 `{1, 2, 3}`，長度是 3。

選 (B) 的人把 `get` 當成 `counts[item]`，後者遇到不存在的鍵才會丟 `KeyError`；選 (C) 的人忘了 set 會去重，直接拿原串列長度 6。

考的觀念：`dict.get` 的預設值、set 自動去重。

W2-16 答 B

列表推導式由右往左讀：先用 `if n % 2 == 0` 留下偶數 2、4、6，再對每一個做 `n ** 2`，得到 `[4, 16, 36]`。

選 (A) 的人把條件看成「餘數不為 0」，留下了奇數；選 (C) 的人只做了篩選、漏了前面的平方；選 (D) 的人漏了篩選條件。

考的觀念：列表推導式的「篩選 → 轉換」順序。

第 3 週 – Adaline 與 NumPy、Pandas

W3-1 答 A

廣播從最右邊（最內層）的維度開始往左對齊比較，每一對維度必須相等、或其中一方為 1；維度數不足的那一方會在左側補 1。符合條件時，NumPy 不會實際複製資料，只是在運算時延展存取。

第三項是常見誤解——`(3, 4)` 與 `(4,)` 維度數不同，照樣可以廣播。第四項也不對：元素總數相同但形狀不相容（例如 `(2, 6)` 與 `(3, 4)`）仍然會失敗。

考的觀念：廣播比的是形狀的對齊，不是元素總數。

W3-2 答 C

`axis` 指的是**被壓縮掉**的維度。對形狀 `(3, 4)` 的陣列做 `sum(axis=0)`，第 0 軸（長度 3）被壓掉，結果形狀是 `(4,)`。

第一項剛好想反了，是最常見的錯誤來源：想對每一列求和時該用 `axis=1`（壓掉欄），而不是 `axis=0`。記住「axis 指定誰消失」就不會搞錯。

考的觀念：`axis` 指定的是消失的維度，不是保留的維度。

W3-3 答 B

`.loc` 依標籤選取，切片**包含**尾端；`.iloc` 依整數位置選取，切片**不包含**尾端，與 Python 原生切片一致。

第三項把兩者對調，是最容易出錯的地方，而且錯了不會報錯——只會安靜地多一列或少一列。`.loc` 之所以包含尾端，是因為標籤沒有「下一個」的概念，寫 `'a':'c'` 時使用者要的就是到 `'c'` 為止。

考的觀念：標籤切片含尾端，位置切片不含。

W3-4 答 D

標準化 $x' = \dfrac{x - \mu}{\sigma}$ 是一個線性變換：平移加縮放。它改變的是位置與尺度，**不改變分布的形狀**——原本是雙峰的標準化後還是雙峰，原本右偏的還是右偏。

第一項是最普遍的誤解，多半來自「z-score」這個名稱讓人聯想到常態分布。第三項描述的是正規化（min-max 縮放），那才會把值壓進 $[0, 1]$。

考的觀念：線性變換動的是位置與尺度，動不了形狀。

W3-5 答 A

全批次每步都要掃完整份資料，方向穩定但單步成本高；隨機梯度下降每筆就更新，快但路徑吵雜、也無法充分利用向量化。Mini-batch 每次取一小批，梯度估計比單筆穩、又能一次向量化處理一批，是實務上的預設選擇。

第三項剛好講反：資料量小的時候全批次本來就負擔得起，mini-batch 的價值正是在大資料集上。

考的觀念：批次大小是在「梯度品質」與「單步成本」之間取捨。

W3-6 答 C

線上學習的重點是「接著學」而不是「重頭學」：權重保留當前狀態，新資料到達就直接更新。這讓模型能隨資料流持續適應，不必為了幾筆新資料付出完整重訓的成本。

第一項正是 `fit()` 的行為，也是 `partial_fit()` 存在的理由——若每次都重新初始化，前面學到的東西就全丟了。

考的觀念：`fit` 重頭訓練，`partial_fit` 接續訓練。

W3-7 答 B

逐筆 Python 迴圈每一次都要付直譯器的開銷；向量化把整個陣列交給預先編譯好的底層常式，用連續記憶體一次處理完，省下的正是那層開銷。

第四項是真的（`ndarray` 固定型別、連續儲存確實更省記憶體），但那是副產品，不是速度的主因。第一項也不對：NumPy 本身跑在 CPU 上。

考的觀念：快的來源是「離開 Python 直譯器」。

W3-8 答 D

`concat` 是「疊」：沿指定軸把多份資料接在一起，對齊靠的是索引。`merge` / `join` 是「配對」：依共同的鍵值把來自兩張表的資料列對應起來，相當於關聯式資料庫的 join。

判斷很簡單——問自己「我是要把更多列（或更多欄）接上去，還是要依某個欄位把兩張表對起來」。前者用 `concat`，後者用 `merge`。

考的觀念：疊起來用 `concat`，依鍵配對用 `merge`。

W3-9 答 A

布林遮罩是先做逐元素比較產生一個布林陣列，再用它篩出為 `True` 的元素；花式索引則是直接把一組索引放進中括號，一次取出指定位置，輸出形狀由索引陣列決定。

第三項兩邊都錯：兩者都回傳複本而非檢視——這一點與基本切片剛好相反，也是「改了卻沒反映回原陣列」這類 bug 的常見來源。

考的觀念：一個用條件選，一個用位置選，兩者都回傳複本。

W3-10 答 C

`np.newaxis` 在索引時插入一個長度為 1 的維度。形狀 `(3,)` 的陣列寫成 `a[:, np.newaxis]` 後變成 `(3, 1)`，就能與 `(3,)` 廣播成 `(3, 3)`——這是計算兩兩距離、外積這類運算的標準手法。

第四項描述的是 `transpose` 或 `swapaxes`。1 這個長度之所以關鍵，正是因為廣播規則允許長度為 1 的維度被延展。

考的觀念：刻意製造長度為 1 的維度，就是刻意製造廣播。

W3-11 答 B

線性激活是恆等函數，它的存在是為了讓損失保留連續的誤差量，梯度才算得出來；閾值函數只在預測階段把連續輸出轉成類別，完全不參與訓練。

第四項若成立，Adaline 就退化回感知器了——閾值函數不可微分，一旦進入梯度計算，整條梯度下降的路就斷了。

考的觀念：訓練看連續誤差，預測才離散化。

W3-12 答 D

單筆樣本估出的梯度只是真實梯度的一個雜訊估計，方向本來就會晃，損失曲線因此毛躁——這是方法的性質，不是錯誤。而這個抖動有個附帶好處：它讓參數不容易卡在淺的局部低點。

第一項與第二項是初次看到這條曲線時最常見的反應。真正該擔心的不是抖動，而是**整體趨勢有沒有往下**；若連趨勢都在上升，那才是學習率的問題。

考的觀念：看趨勢，不看單點的抖動。

W3-13 答 B

`axis` 指的是**被壓扁**的那一軸。`sum(axis=0)` 壓掉長度 3 的第 0 軸，剩下 `(4,)`，也就是每一欄的總和；`mean(axis=1)` 壓掉長度 4 的第 1 軸，剩下 `(3,)`，是每一列的平均。

選 (A) 的人把 `axis` 理解成「保留的那一軸」，剛好顛倒；選 (C) 的人以為聚合後維度數不變，但預設會直接拿掉被壓扁的軸（要保留得另外加 `keepdims=True`）。

考的觀念：`axis` 是被壓扁的維度，聚合後該軸消失。

W3-14 答 D

NumPy 的 `*` 是**逐元素**相乘：對應位置各自相乘，`B` 的非對角線是 0，所以結果是 `[[1 0] [0 4]]`。`@` 才是線性代數的矩陣乘法，`B` 是單位矩陣，$AB = A$，結果是 `[[1 2] [3 4]]`。

選 (C) 的人把兩個運算子的意義對調了，這正是最常見的混淆；選 (A)、(B) 的人以為 `*` 與 `@` 是同一件事。

考的觀念：`*` 逐元素、`@` 矩陣乘法。

W3-15 答 A

廣播先把較小者的形狀**從左邊補 1**，再由右往左逐維比對：相等或其中一邊是 1 才行。`v` 是 `(4,)`，補成 `(1, 4)`，與 `(3, 4)` 對齊，可以運算。`w` 是 `(3,)`，補成 `(1, 3)`，最後一維 3 對 4、兩邊都不是 1，報錯。

選 (C) 的人以為補 1 是補在右邊、讓 `(3,)` 對上列數 3；要讓 `w` 對到每一列，得先把它變成 `(3, 1)`（例如 `w[:, np.newaxis]`）。選 (D) 忽略了廣播的存在。

考的觀念：廣播從右邊對齊、從左邊補 1。

W3-16 答 C

`.loc` 依**標籤**選取，切片**包含**尾端，`0:2` 取到標籤 0、1、2 共 3 列；`.iloc` 依**整數位置**選取，行為和 Python 切片一樣**不含**尾端，`0:2` 只取位置 0、1 共 2 列。

選 (B) 的人把兩者的規則對調；選 (A) 的人以為兩者都照 Python 切片規則。這裡索引剛好是 0、1、2、3，標籤和位置數值相同，更容易誤以為兩者等價。

考的觀念：`.loc` 含尾端、`.iloc` 不含尾端。

第 5 週 – 邏輯迴歸、Matplotlib 與 Seaborn

W5-1 答 A

$\text{odds}(p) = \dfrac{p}{1-p} = \dfrac{0.8}{0.2} = 4$，意思是發生的可能性是不發生的 4 倍。

第二項把勝算跟機率當成同一個量，是最常見的混淆。兩者的值域也不同：機率落在 $(0, 1)$，勝算落在 $(0, \infty)$——而再取對數就得到 logit，值域擴張到整條實數線，這正是它能被線性模型建模的原因。

考的觀念：機率、勝算、對數勝算是三個值域不同的量。

W5-2 答 C

邏輯迴歸令 $z = \boldsymbol{w}^T\boldsymbol{x} + b = \text{logit}(p)$，線性的是對數勝算；機率則由反函數 sigmoid 得到，$p = \sigma(z)$。

第一項是最普遍的誤解。直接對 p 做線性迴歸會出問題：線性函數的值域是整條實數線，而機率必須落在 $(0, 1)$，預測值很容易跑到區間外。先把 p 映射到實數線再建模，正是為了避開這件事。

考的觀念：先擴張值域再線性建模，最後用反函數映射回機率。

W5-3 答 B

sigmoid 是單調函數，判斷 $\sigma(z) \geq 0.5$ 等價於判斷 $z \geq 0$，而 $z = \boldsymbol{w}^T\boldsymbol{x} + b$ 是線性的——決策邊界仍是超平面，sigmoid 改變的只是輸出的刻度（從分數變成機率）。

第一項是最誘人的錯誤：非線性函數確實進了模型，但它作用在分數上、不作用在特徵上。要讓邊界真的彎曲，得從特徵下手（例如加入 $|x_1|$ 或交互項），這就是特徵工程在做的事。

考的觀念：非線性加在哪裡，決定邊界會不會彎。

W5-4 答 D

概似把資料固定下來，問「哪一組參數讓這些標籤出現得最合理」——它是參數的函數。最大概似估計就是在參數空間上找這個函數的最大值。

第三項的誤解特別常見：概似的值雖然由機率相乘而來，但它不是把參數當成隨機事件在算機率，也不需要對參數積分為 1。二元交叉熵正是 Bernoulli 模型的平均負對數概似，最小化它就等於最大化概似。

考的觀念：機率固定參數看資料，概似固定資料看參數。

W5-5 答 A

BCE 的每一項是 $-\ln(a)$ 或 $-\ln(1-a)$。預測 0.99 而真實為 0 時，損失是 $-\ln(0.01) \approx 4.6$；預測 0.49 而答錯時只有 $-\ln(0.51) \approx 0.67$。對數在接近 0 的地方急遽發散，於是「自信地答錯」的代價遠高於「猶豫地答錯」。

這個性質是刻意的：它逼模型在沒把握時輸出接近 0.5，而不是隨便給一個極端值賭運氣。

考的觀念：對數損失懲罰的是「錯得多有自信」，不只是「錯沒錯」。

W5-6 答 C

線性模型對 x_1 的反應是單調的：x_1 越大分數越高（或越低），沒辦法表達「太大或太小都不好、接近 0 才正常」這種對稱條件。換成 $|x_1|$ 之後，偏離 0 的程度成為一個單調的量，線性模型就能用它來表達這個規則。

晶圓驗收這類「量測值偏離規格中心就判不合格」的任務正是這個形狀。第一項剛好相反——取絕對值丟掉的就是正負號。

考的觀念：特徵工程是把模型表達不了的形狀，轉成它表達得了的形狀。

W5-7 答 B

Figure 是整張畫布，一張 Figure 可以包含多個 Axes；每個 Axes 是一整張子圖（含它自己的 x 軸與 y 軸），不是「一條軸」。

名稱是主要的誤解來源：`Axes` 看起來像 `axis` 的複數，但 Matplotlib 裡的 `Axis` 才是單一座標軸。搞混這一層，`fig, ax = plt.subplots(2, 3)` 回傳的東西就讀不懂了。

考的觀念：Axes 是子圖，Axis 才是座標軸。

W5-8 答 D

`plt.plot` 加上標記參數一樣能畫散佈圖，所以差別不在圖形外觀。真正的差別是 `plt.scatter` 允許**每個點長得不一樣**：`s` 控制大小、`c` 控制顏色、`alpha` 控制透明度，因此一張二維圖能塞進四個變數。

代價是速度：`plt.plot` 全部共用一組樣式，畫十萬個點很快；`plt.scatter` 點多時明顯較慢。所以第三項恰好講反了。

考的觀念：能不能逐點變化，是兩者的分界。

W5-9 答 A

箱寬是直方圖最關鍵的旋鈕。太寬會把相鄰的細節合併進同一格，雙峰因此被抹平成單峰，讀者看到的是一個不存在的結論；太窄則每格樣本太少，整張圖都是抖動——那是第二項描述的情況。

第三項不對：分箱只改變統計的分組方式，樣本總數始終不變。實務上該做的是換幾個箱寬看看結論穩不穩定。

考的觀念：箱寬太寬抹掉結構，太窄放大雜訊。

W5–10　答 C

發散型色盤從中點往兩側各用一種色系加深，正好對應「高於或低於某個基準」的資料（溫度距平、盈虧、殘差）。連續型適合單向遞增的量，質性色盤用於無序類別。

第四項要特別避開：彩虹型色盤（如 jet）的亮度起伏不單調，轉成灰階後順序整個亂掉，對色覺障礙讀者也不友善。挑完色盤轉灰階檢查一次是個好習慣。

考的觀念：色盤類型要對應資料的結構，而不是好不好看。

W5–11　答 B

`transAxes` 用的是子圖內部的相對座標：$(0, 0)$ 是左下角、$(1, 1)$ 是右上角，因此不論資料範圍怎麼變，文字都待在同一個相對位置——想把說明固定貼在角落時用它。

`transData`（第一項）用資料座標，文字會跟著資料跑，適合指向特定資料點；`transFigure`（第三項）則相對整張畫布，用於跨子圖的標註。

考的觀念：先決定「文字要跟著誰動」，再選座標系。

W5–12　答 D

Seaborn 架在 Matplotlib 之上，API 更高階、原生吃 DataFrame、還把統計動作包了進去——剛拿到資料想快速掃過分布與相關性時，一行 `pairplot` 的產值遠高於自己刻迴圈。等圖要進報告、需要精準控制標註位置、色條刻度與子圖對齊時，就回到 Matplotlib 的 Axes 物件慢慢調。

兩者不是取代關係：Seaborn 畫完也回傳 Axes，可以無縫接手。這也對應視覺化的兩種角色——探索式畫給自己看，要快、要多、丟得掉；解釋式畫給別人看，一張圖只准講一件事。

考的觀念：判準是「這張圖畫給誰看」。

W5–13　答 D

邏輯迴歸保留 Adaline 的淨輸入與梯度下降骨架，只換兩個零件：激活函數換成 sigmoid，損失換成**二元交叉熵**。二元交叉熵就是把概似函數取對數、加負號後的結果，所以也叫負對數概似。

選 (A) 最誘人：MSE 是 Adaline 的損失，邏輯迴歸正是因為 MSE 不適合分類才換掉它。選 (B) 是感知器的想法，它根本沒有可微分的損失，只看分錯才更新。

考的觀念：模型與損失函數的配對——感知器無損失、Adaline 用 MSE、邏輯迴歸用二元交叉熵。

W5–14　答 B

三個模型的淨輸入都是 $z = \boldsymbol{w}^T \boldsymbol{x} + b$，差在激活函數：感知器用階梯函數，直接輸出類別；Adaline 用恆等函數，輸出任意實數，訓練時拿它算 MSE，分類時才切門檻；邏輯迴歸用 sigmoid，輸出正類的條件機率，通常以 0.5 切分。

(A) 把邏輯迴歸的激活函數安到感知器上；(C) 把感知器的階梯函數安到邏輯迴歸上；(D) 描述的其實是 Adaline。

考的觀念：三個線性分類器的激活函數與輸出意義。

W5–15　答 C

MSE 只看 z 與標籤差多少，不管方向對不對。$z = 5$ 離標籤 1 差 4，平方後損失是 16，優化器會想把 z 拉回 1，結果把決策邊界往錯的方向拉。這就是講義說明 Adaline 不適合分類、要改成邏輯迴歸的理由：sigmoid 把輸出壓在 0 到 1 之間，分得越正確，交叉熵損失越接近 0。

選 (A)、(B) 的人把「分類正確」直接等同於「損失很小」，那是交叉熵的性質，不是 MSE 的。選 (D) 混淆了感知器的更新規則。

考的觀念：為什麼邏輯迴歸不用 MSE。

W5–16　答 A

這是**多標籤**問題：同一筆資料可以同時屬於好幾類。OvR 為每個類別各自訓練「這一類 vs 其他」的分類器，各自輸出機率、互不牽制，所以可以同時判定兩種病變都有。

選 (B) 最誘人：softmax 把所有類別的機率綁成總和 1，最大的那一類必然壓低其他類，適合「多選一」的互斥類別，不適合「可能同時有好幾種」。(C)、(D) 都只能輸出一類。

考的觀念：多類別（多選一）用 softmax，多標籤只能用 OvR。

第 6 週 – Scikit-learn 機器學習

W6-1 答 C

Adaline 只改一處：更新權重時不看跳階函數的對錯，而看連續的線性輸出，用平方誤差當損失。損失一旦可微，就成了一個碗狀曲面，訓練於是變成梯度下降。

最容易誤選的是第一項——sigmoid 與對數損失是邏輯斯迴歸的做法，不是 Adaline。第二項描述的是隨機梯度下降，那是訓練方式的差別，不是模型定義的差別；第四項是正則化。

考的觀念：損失可微才談得上梯度下降。

W6-2 答 B

k 近鄰是懶惰學習者，訓練時只把資料記住，預測時才要跟每一個樣本比距離；核支援向量機的訓練要解二次規劃（昂貴），但預測時只跟少數支援向量比（便宜）。

第三項剛好把兩者對調，也是最容易記反的地方。實務上的差別很直接：上百萬筆資料時核支援向量機的訓練成本會爆掉，要改用支援 `partial_fit` 的隨機梯度下降；k 近鄰則相反，訓練免費但每次預測都要翻過全部資料。

考的觀念：成本會從哪一端付出去，決定模型適不適合你的場景。

W6-3 答 D

正確順序是切分 → 只在訓練集上 `fit` 轉換器 → `transform` → 測試。

第三項最容易被誤選，因為它確實「有先切分」，看起來已經守住規則；但對測試集呼叫 `fit_transform` 等於用測試集自己的統計量另建一套座標系，兩邊不再是同一把尺量出來的，模型看到的分布也與訓練時不一致。第一項與第二項則都讓測試集的統計量參與了轉換器的建立，是典型的資料洩漏，後果是測試分數偏高、上線就掉。

考的觀念：資料洩漏的判準是「測試集的資訊有沒有參與訓練流程」。

W6-4 答 A

這是一個兩難：測試集越大，學習演算法能用的資訊越少；測試集越小，泛化誤差的估計越不準。第二項與第三項各把兩難的一邊講反了。

因為是兩難，就沒有普世最佳解——常用的 60:40、70:30、80:20 取決於原始資料量，所以第四項也不對。

考的觀念：切分比例是在「可用資訊」與「估計精度」之間取平衡。

W6-5 答 C

`isnull()` 只對 `NaN`、`None`、`NaT` 回傳 `True`；空白字串 `""` 算有效資料。

第一項是最常見的誤判：肉眼看起來「那一格是空的」，但它在資料結構裡是一個長度為零的字串，不是缺值。實務上的後果是 `isnull().sum()` 數出來的缺值數量偏低，補值步驟漏掉那些欄位，問題要到模型階段才浮現。

考的觀念：「看起來是空的」與「資料結構上是缺值」是兩件事。

W6-6 答 B

類別標籤不使用順序資訊，指派哪個整數給哪個字串標籤都無所謂——scikit-learn 的分類估計器內部本來就會把標籤轉成整數。

第四項是最容易被誤選的，因為它把「特徵」的規則搬到了「標籤」上：順序型與名義型的區分、以及要不要獨熱編碼，處理的都是輸入特徵 `X`，不是標籤 `y`。

考的觀念：特徵的編碼規則不會自動適用於標籤。

W6-7 答 A

講義特別提醒：獨熱編碼後產生的 0 與 1 特徵欄位不需要再進行特徵縮放。

第三項是自我矛盾的選項——正規化的目的就是把數值壓到 $[0, 1]$，而這些欄位已經只有 0 與 1 兩個值，再壓一次不會有任何改變。第二項聽起來合理（「尺度要一致」），但 0 / 1 欄位的尺度已經落在其他標準化欄位的合理範圍內，硬做標準化反而會讓「有 / 沒有」這個二元意義變成不好解讀的正負數。

考的觀念：縮放是為了讓尺度可比，本來就可比的欄位不必再動。

W6-8 答 D

正規化是 min–max 縮放，分母是該欄的極差；單一個極端離群值就會把 $x_{\max}$ 拉得很遠，其餘資料因此被壓縮進 $[0, 1]$ 裡極小的一段。標準化用平均與標準差，保留了離群值的資訊，也讓演算法對它較不敏感，對梯度下降與隨機梯度下降的最佳化尤其實用。

第二項把兩者對調了，是最容易記反的一組。要注意兩者都不會剔除離群值（第一項）——想抗離群值要換 `RobustScaler`，而需要有界區間時才選正規化。

考的觀念：min–max 的分母是極差，所以它對極值敏感。

W6-9 答 C

貪婪演算法在每一階段做局部最優選擇，因此通常得到次佳解——這正是它換來可行計算量所付的代價。

第一項與第四項描述的是窮舉搜尋：窮舉會評估所有組合並找到最佳解，但組合數上完全不可行。第二項則把「組合爆炸」這個窮舉的缺點錯掛在貪婪法上，方向剛好相反。

考的觀念：貪婪法用「可能不是最好」換到了可行的計算量。

W6-10 答 B

過擬合的定義要求兩件事同時成立：訓練損失已經夠低，而驗證損失跟不上。這題連訓練損失都更高，第一個條件就不成立。

參數多的模型能表達的函式集合包含了參數少的模型能表達的一切，理論上訓練損失至少該一樣低；既然沒有，問題出在「沒有找到那個更好的解」，也就是最佳化。第一項是最常見的誤判，而它的處方（正則化）與真正需要的處方（改善最佳化）方向完全相反。

考的觀念：訓練損失本身降不下來，不叫過擬合。

W6-11 答 D

正則化等價於在約束區域內找均方誤差最小的點。L_2 的約束區域是圓、L_1 是菱形，而菱形的四個角剛好長在座標軸上；等高線由內往外擴張時很容易先碰到角，那個位置某一個權重恰好為 0。

差別不在懲罰多重（第一項）——同樣的 λ 下兩者的懲罰量並沒有固定的大小關係。第三項描述的是門檻化這種後處理操作，不是正則化的機制：L_1 的零不是被裁掉的，而是最佳解本來就落在軸上。

考的觀念：約束區域的形狀有沒有角，決定解會不會被吸到座標軸上。

W6-12 答 A

同一個 Pipeline 物件有兩條路徑：`fit()` 時前面數個轉換器依序 `fit_transform`、最後一個估計器 `fit`；`predict()` 時資料只依序通過各轉換器的 `transform`，再交給估計器 `predict`。

第二項與第三項是最常見的誤解，而它們恰恰是 Pipeline 存在的理由——它讓「轉換器只能用訓練資料 `fit`」變成結構上的保證，而不是靠人記得。第四項會讓估計器吃到未經縮放的原始資料。

考的觀念：預測路徑上只有 `transform`，沒有 `fit`。

W6-13 答 A

標準化要用的平均與標準差只能從訓練集算。正確寫法是第 4 行改成 `sc.transform(X_test)`，沿用訓練集 fit 出來的參數。講義的原則是：絕對不要在測試資料上呼叫 `fit()` 或 `fit_transform()`，否則測試集的統計量混進流程，分數虛高、上線就掉。

選 (D) 最誘人：「各自標準化」聽起來公平，但兩份資料被放到不同的座標系，同一個原始數值在兩邊會變成不同的值。選 (B) 是另一種洩漏，違反「先切分、再處理」。(C) 弄反了：`fit_transform` 正是轉換器的方法。

考的觀念：轉換器只對訓練集 fit，測試集只 transform。

W6-14 答 C

scikit-learn 的慣例是：`fit()` 學到的結果一律放在**結尾帶底線**的屬性。`coef_` 是權重 $\boldsymbol{w}$，`intercept_` 是偏置項 b。建構子裡的 `eta0`（學習率）、`random_state` 是超參數，名字不帶底線。

選 (A)、(B) 的人分不清「人先設的超參數」與「從資料學到的參數」；(D) 描述的是 `StandardScaler` 學到的 `mean_` 與 `scale_`。

考的觀念：結尾底線的屬性是 fit 之後學到的參數。

W6–15　答 B

凡是靠距離或靠梯度的模型都怕尺度不一：KNN 算距離時，數值大的特徵會主導結果；SVM 的間隔同樣以距離衡量；邏輯迴歸靠梯度下降，尺度差太多會收斂很慢，加上正則化時也會不公平地懲罰權重。

選 (A) 最誘人但弄反了：樹模型每次只拿一個特徵比大小決定切在哪，縮放不會改變排序，所以不需要。(C) 忽略了梯度下降與正則化。(D) 低估了縮放的影響：對 KNN 而言，縮放會改變預測結果本身，不只是速度。

考的觀念：距離與梯度類模型要縮放，樹模型不需要。

W6–16　答 D

`stratify=y` 是**分層**切分：依標籤 `y` 的類別比例抽樣，讓訓練集與測試集裡各類別所佔的比例都和原始資料一樣，避免某一類在測試集裡偏多或偏少。

(A) 是 `random_state=1` 的作用；(B) 是 `test_size=0.3` 的作用。這兩個參數就在同一行，最容易張冠李戴。(C) 剛好相反：依類別排序後切段，正是 Iris 那種「按類別排好」的資料最怕的事。

考的觀念：`train_test_split` 各參數的功能。

第 7 週 – 資料前處理、特徵工程與模型選擇評估

W7–1　答 B

狹義是超參數調整——對指定的模型或演算法，選出調校參數的最佳值；廣義是演算法選擇——比較不同演算法各自的最佳表現，挑出最好的那個。

第一項把兩者對調。第四項把驗證架構也講反了：狹義的調參用單層交叉驗證就夠，廣義的演算法比較才需要巢狀交叉驗證。

考的觀念：先分清楚在選什麼，才知道該用哪一套驗證架構。

W7–2　答 C

holdout 只切一次，換一種切法結論可能就變了——k 折交叉驗證正是為了降低這個敏感度而生。

第一項描述的不是 holdout 的定義問題：它本來就切出訓練、驗證、測試三份，測試集是獨立的。真正要小心的是驗證集與測試集不能是同一份——驗證集參與了選擇，拿它報告泛化效能會偏樂觀。第二項是 k 折的成本特性，不是 holdout 的。

考的觀念：單次切分的脆弱性。

W7–3　答 A

是反向關係：資料量小時用大的 k，每一折留下的訓練資料才夠多；資料量大時用小的 k 省計算成本，估計本來就夠穩。

第三項是最容易誤選的——$k = 10$ 是常用的**標準值**，不是固定值；另一個極端是 k 等於訓練樣本數，也就是留一交叉驗證。維持每折類別比例的是分層 k 折，與 k 的大小無關（第四項）。

考的觀念：k 是在「每折的訓練資料量」與「計算成本」之間取捨。

W7–4　答 D

我們要的是單一最終模型，而不是那 k 個個別模型；訓練樣本越多，模型通常越準也越穩健。

第一項是最常見的誤解——那 k 個模型各自只看過 $k - 1$ 折的資料，挑分數最高的那一個，等於挑到「剛好遇到好切分」的那一個。第三項則直接違反「測試集只在最後動一次，用來確認、不用來挑選」。

考的觀念：交叉驗證產出的是效能「估計」，不是最終模型。

W7–5　答 B

兩線都低且貼合是高偏差、欠擬合。

第一項是這張圖最想破除的誤解：補資料能拉近的是「兩線之間那道持續的縫」，而這裡根本沒有縫，再多資料也只是把同一條平線延長。第三項是高變異（過擬合）的處方，用在欠擬合上會讓模型更學不動，方向剛好相反——同一張圖上兩種病徵的處方是相反的，認錯了會越調越糟。

考的觀念：學習曲線真正回答的是「該不該再去要資料」。

W7–6　答 C

驗證分數的峰值就是該選的值；右端訓練分數續升而驗證掉頭，正是過擬合開始的位置。

第一項是最誘人的錯誤：訓練分數往往隨模型變複雜而單調上升，照它挑必定挑到過擬合的那一端。第二項描述的是縫最大處，那同樣是過擬合最嚴重的地方。

考的觀念：挑超參數看的一律是驗證分數，不是訓練分數。

W7-7 答 D

格狀搜尋被迫在每個維度上都試同樣多次，於是在不重要的維度上重複浪費；隨機搜尋用同樣的預算，能在真正重要的那一軸上取樣得更密。

第一項剛好講反——保證找到最佳解（只要它在網格內）的是格狀搜尋，隨機搜尋沒有這個保證。第三項描述的是逐次減半搜尋：先用少量資源評估所有候選，反覆淘汰並把資源集中到存活者。

考的觀念：差別在預算怎麼分配，不在單次評估的成本。

W7-8 答 A

外層 k 折切出訓練折與測試折、只負責評估；內層在外層的訓練折上再跑一次 k 折，用來選超參數。測試折從頭到尾沒有參與選擇，得到的才是不偏的估計。

第二項把分工對調，等於讓測試折參與了挑選，分數會樂觀——那正是單層交叉驗證的問題，也是巢狀交叉驗證存在的理由。講義的範例是外層 5 折、內層 2 折的 5×2 交叉驗證。

考的觀念：評估與挑選必須用不同的資料。

W7-9 答 C

調和平均對較小的那一個特別敏感，所以只有精確率與召回率兩邊都高，F1 才會高。

具體看差別：若精確率 1.0、召回率 0.0，算術平均給出 0.5 這種看起來還行的分數，但那個模型其實一個陽性都沒抓到；調和平均在這裡是 0。第二項不成立——算術平均的值域同樣是 0 到 1。

考的觀念：F1 要懲罰的正是「一邊很高、另一邊很低」。

W7-10 答 B

門檻調鬆後判為陽性的樣本變多，真陽性被抓到的比例（召回率）上升；但誤報也跟著變多，判為陽性的樣本中真的是陽性的比例（精確率）下降。

第三項是最直覺的誤判——「判為陽性的變多」確實讓召回率上升，但精確率的分母 $TP + FP$ 也跟著變大，兩者是拉鋸而不是同向。實務上的取捨由代價結構決定：癌症篩檢寧可誤報不能漏報，優先召回率；垃圾信過濾誤殺重要信的代價高，偏向精確率。

考的觀念：精確率與召回率的拉鋸來自同一個門檻。

W7-11 答 D

F1 由精確率與召回率合成，而這兩者都不看真陰性 TN；MCC 的公式同時吃 TP、TN、FP、FN 四格，因此比較不會被單邊表現騙過。

第一項把值域範圍當成理由——值域寬不等於不易被誤導。第三項剛好講反：MCC 正是因為沒有忽略多數類才可靠。第二項也不對，MCC 與 F1 一樣是綁在某一個決策門檻上的指標，不受門檻影響的是 ROC AUC。

考的觀念：一個指標可不可靠，看它用了四格中的哪幾格。

W7-12 答 A

是雙底線 `__` 語法，例如 `logisticregression__C`。

第三項在單一估計器時行得通，但 Pipeline 裡可能有多個步驟帶同名參數（例如兩個轉換器都有 `strategy`），沒有前綴就無法區分是哪一個。第四項技術上做得到，卻會讓格狀搜尋無法把該參數納入搜尋空間——`GridSearchCV` 的參數網格正是靠這套命名定位到 pipeline 內部的步驟。

考的觀念：`__` 是 scikit-learn 用來定位巢狀物件參數的通用語法。

W7-13 答 D

`cv=10` 是 10 折交叉驗證：資料分成 10 份，輪流拿其中 1 份當驗證、其餘 9 份訓練，共跑 10 輪，每輪得到一個驗證分數，所以回傳長度 10 的陣列。要得到平均，得自己再呼叫 `np.mean(scores)`。

選 (A) 的人以為函式會幫你平均；選 (B) 的人把「每輪用幾折訓練」和「共跑幾輪」搞混；(C) 是留一法交叉驗證（LOOCV）的情況。

考的觀念：`cross_val_score` 回傳每一折的分數，個數等於 k。

W7–14　答 A

格狀搜尋是**窮舉**所有參數組合：`C` 有 4 個值、`gamma` 有 3 個值，共 $4 \times 3 = 12$ 組；每一組都要做 5 折交叉驗證，所以是 $12 \times 5 = 60$ 次訓練。

選 (B) 的人只算了組合數，忘了每組要跑 5 折；選 (C) 的人把兩個參數的值數相加（$4 + 3 = 7$）再乘 5；選 (D) 的人只拿 `C` 的 4 個值乘 5。這也說明格狀搜尋的成本隨參數個數乘法成長，參數一多就改用隨機搜尋。

考的觀念：格狀搜尋的訓練次數 = 參數組合數 × 折數。

W7–15　答 C

`best_estimator_` 是格狀搜尋挑出的最佳參數組合所對應的模型。這段程式用訓練集訓練它，再用 `score` 在**測試集**上評估一次。這正是工作流程的最後一步：超參數全部決定後，測試集才拿出來用一次。

選 (B) 的人把測試分數和交叉驗證分數搞混，這裡 `score` 吃的是 `X_test`。(A)、(D) 都是拿測試集來調模型，違反「測試集全程不碰」，測出來的分數會過度樂觀。

考的觀念：最佳模型只在測試集上評估一次。

W7–16　答 B

逐筆對照：第 1 筆真 1 預 1 是 TP；第 2、3 筆真 1 預 0 是 FN；第 4 筆真 0 預 1 是 FP；第 5、6 筆是 TN。所以 TP=1、FN=2、FP=1。

$$\text{PRE} = \frac{TP}{TP + FP} = \frac{1}{2} = 0.50$$

$$\text{REC} = \frac{TP}{TP + FN} = \frac{1}{3} \approx 0.33$$

選 (A) 的人把精確率和召回率的分母對調了：精確率問「預測為陽性的裡面有多少是真的」，召回率問「真的陽性裡面抓到多少」。

考的觀念：從預測結果數出 TP / FP / FN，再套精確率與召回率。
